

Chapter 2. Architectures and Patterns: Planning, Reactivity, and Multi-Agent-Systems

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at mcronin@oreilly.com.

You know by now that AI agents are not magic. They are engineered systems whose power comes from the strategic configuration of their architecture. By combining reasoning loops, tools, stateful design, and adaptive control, LLMs can move beyond static prompts and evolve into MAS that plan, react, and reflect in real time. This chapter explores the architectures that make this possible, from structured reasoning methods such as chain of thought, tree of thought, and ReAct to architectural paradigms like supervisor and hierarchical agents. Along the way you will gain a deeper understanding of reasoning flows and see how human-in-the-loop mechanisms including approval gates, corrections, and interruptions keep these systems reliable, aligned, and secure.

The importance of an agent’s architecture becomes obvious when you compare how a simple single-step agent handles a task versus how a multi-step agent does. A single-step agent receives the request, plans everything internally, executes in one go, and returns the result without ever revisiting intermediate steps. A multi-step agent, in contrast, alternates between reasoning and action in an iterative loop, adapting its plan based on partial results or new observations as it works. This flexibility is what allows agents to refine outputs dynamically and cope with uncertainty or feedback from their environment. The contrast is illustrated in [Figure 2-1](#), with [Table 2-1](#) showing how each approach unfolds step by step on the same task.

Table 2-1. Example: Single-step vs. Multi-step (ReAct) execution for the task “Summarize the document and create a bar chart of key statistics”

Step	Single-step agent	Multi-step (ReAct) agent
1	Reads the request once	Reads the request and forms an initial thought on the first action (extract statistics)
2	Plans the entire sequence internally	Action: execute extraction; observation: store extracted data in memory
3	Generates summary and chart in a single pass	Thought: plan next action to generate the summary from extracted data
4	Returns outputs without revisiting intermediate results	Action: generate summary; Observation: store output in memory
5	No mid-execution adjustments	Thought: plan final action to create the bar chart; action: create chart; observation: review output
6	—	If intermediate results are incomplete, repeat the loop until satisfied, then return final outputs

While the table shows the difference step by step, [Figure 2-1](#) illustrates the overall workflows side by side. The single-step agent follows a linear pipeline from prompt to result, whereas the multi-step agent embeds reasoning, actions, and memory into an iterative loop that continues until the problem is solved or the computational budget is exhausted.

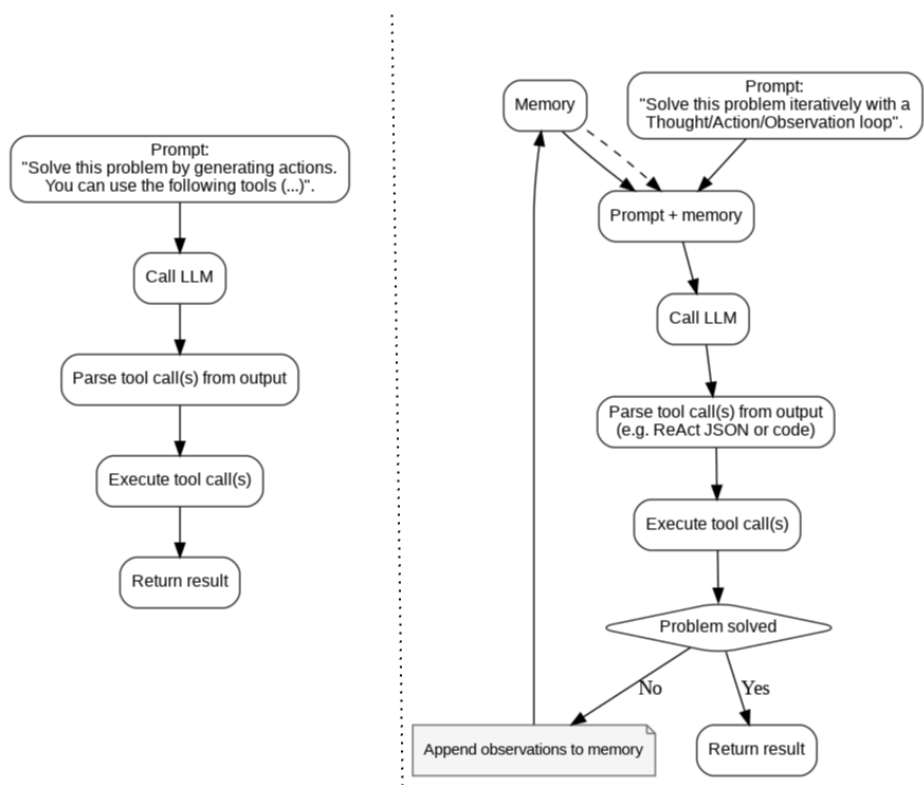


Figure 2-1. Single-step (left) versus ReAct (right) agent workflows.

Structured Reasoning and Action

Giving agents distinct personas, sometimes called anthropomorphizing, can sharpen their reasoning and make their contributions more focused. A planner agent framed as a methodical strategist will approach problems differently than a critic agent framed as an evaluator. When each agent is given a clear role and mindset, their reasoning chains tend to be more coherent and their interactions in MAS more purposeful. This alignment improves collaboration, since each agent knows what kind of knowledge it's expected to contribute.

Yet defining a role is only half the picture. To make agents more effective you might also need to shape how the agent thinks. This is where structured reasoning methods such as chain of thought, tree of thought and ReAct come into play.

Letting Your AI Agents Think Out Loud

Chain-of-thought prompting (CoT) ¹ is inspired by the human thought process, this technique first solves intermediate steps before getting to the final answer. While CoT gets often introduced as prompting trick, in the context of AI agents CoT can serve as added reasoning steps that guides control flow. By forcing the model to generate intermediate steps, you gain better transparency over the reasoning process. It often improves reliability and can help smaller models perform tasks that would otherwise require a larger model. This keeps cost lower while providing clearer insights into how the agent arrives at its decisions. [Table 2-2](#) provides you ideas for prompts for common scenarios that you can adapt to your needs.

Table 2-2. Examples: Chain of Thought prompts by scenario

Scenario	CoT prompt example
Planning	“Plan step by step to create a one-week blog schedule. Explain the reasoning behind each choice.”
Data analysis	“Begin by describing the dataset. Next highlight patterns or trends. Conclude with a clear summary of insights.”
Math problem	“Break the problem into smaller pieces. Solve each piece with explanation. Then combine the results for the final answer.”
Research task	“List the central questions first. Investigate them one by one. Conclude with a synthesized summary.”
Debugging	“Start by listing possible causes. Test each cause in turn. Eliminate what does not fit and explain why until you reach the fix.”
Decision-making	“List all available options. Consider pros and cons step by step. Choose the option that best fits the criteria and explain why.”

To make this more explicit how you might want to implement this into your workflow, let's look at an agent for data analysis. Since the core of a CoT agent is its ability to reason in a series of logical, intermediate steps, this is the perfect example. You can instruct the agent to think step by step, and have it begin by describing first the dataset and next have it highlight patterns or trends. [Example 2-1](#) shows a generic system prompt you could use for such an agent. I omit the rest of the code for brevity, but you can find the complete working implementation in the [book's repository](#).

Example 2-1. CoT prompt

```
SYSTEM_INSTRUCTIONS = """You are a careful data analysis assistant. Think step by step and be explicit.
Begin by describing the dataset.
Next highlight patterns or trends.
Conclude with a clear summary of insights.
When you need to compute metrics or create a chart, call the python_repl tool.
code that:
1. Loads the CSV from the dataset path
2. Prints descriptive statistics
3. If a target column exists, creates one bar chart of mean values grouped by target and saves it to a file
4. Prints key results
Return FINAL ANSWER only after you have executed your analysis."""
```

Now to use the prompt with your agent, you just need to create a single agent as show in [Example 2-2](#). Note that I use the `AgentExecutor` from

LangChain together with LangGraph for brevity in [Example 2-2](#). But in the accompanying notebook you'll also find a more granular LangGraph wiring, where the routing between the model and tools is made explicit.

Both approaches yield the same result. However, using the `AgentExecutor` is often preferable in production, since it keeps your code concise, while the granular graph is valuable for debugging and for understanding how messages, tool calls, and control flow work under the hood.

Example 2-2. CoT Agent

```
def create_agent(llm: ChatOpenAI, tools: list, system_prompt: str) -> AgentExecutor:
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", system_prompt + """Work autonomously using the available
            tools."""),
            MessagesPlaceholder(variable_name="messages"),
            MessagesPlaceholder(variable_name="agent_scratchpad"),
        ]
    )
    agent = create_openai_functions_agent(llm, tools, prompt)
    executor = AgentExecutor(
        agent=agent,
        tools=tools,
        return_intermediate_steps=True,
        handle_parsing_errors=True,
        verbose=False,
    )
    return executor
```

When I ran the code, the agent produced a correct description of the demo dataset, an accurate analysis of the patterns and trends, as well as a bar chart and a summary of its insights. In addition, with the provided code, you can inspect the intermediate tool steps, for example:

```
(AgentActionMessageLog(tool='python_repl', tool_input={'code': "import pandas
as pd\n\ndf = pd.read_csv('demo.csv')\n\ndf.describe()"}))
```

or

```
AgentActionMessageLog(tool='python_repl', tool_input={'code': 'df.describe()'})
```

This example illustrates how CoT enables the agent to make its reasoning process visible and verifiable. If you use the code from this example, you basically force the agent to externalize its intermediate steps, and with that you not only gain confidence in the results but also create a transparent audit trail of how conclusions are reached.

Generating a Tree of Possibilities

Now think of a scenario where you have a more complex task, but you don't want to use a reasoning model or a full multi-agent architecture. Still, a more exploratory and strategic approach is needed. In such cases, you can use *tree of thoughts prompting* (ToT) ². ToT expands CoT reasoning into a branching search process, where multiple reasoning paths are explored and evaluated before deciding on the best next step.

In the code examples ([Example 2-3](#) - [Example 2-5](#)), I'll show you how to implement ToT. The key idea is to let a smaller, single AI agent generate multiple potential paths (a “tree”), have another agent evaluate and select the best one, and then execute that chosen path with yet another model. The simplified ToT method offers a pragmatic middle ground between a simple ReAct loop (see [“How to Make Your Agent Take Action and Reflect”](#)) and the full computational complexity of more tree-search-based algorithms, which I will cover in the next chapter.

TOT AS AN ENGINEERING SHORTCUT

A lightweight ToT setup where a smaller model proposes branches and a stronger model evaluates can sometimes replace the need for a full multi-agent architecture. This simplifies design and saves both money and time by avoiding repeated calls to large reasoning models when a smaller setup already provides reliable results.

You begin by creating a “tree” of possibilities. The `propose_options` node in [Example 2-3](#) is the first step. Instead of producing a single, linear output, it uses a smaller LLM to generate three distinct approaches for a blog post. These three options represent branching paths, or “thoughts,” that the agent can explore. This is the core principle of ToT: moving beyond a single-chain process toward multi-path exploration.

Example 2-3. Thought Generation

```
def propose_options(state: BlogState) -> BlogState:
    """Generator (small): propose 3 creative approaches (ToT-style branching)
    proposer = gen_llm.with_structured_output(OptionsPayload)
    prompt = (
        "Generate exactly 3 distinct approaches for a developer-focused blog
        f"{state['topic']}\n\nFor each option, provide: title, audience, angle,
        "a 5-bullet outline, and a concise rationale."
    )
    payload: OptionsPayload = proposer.invoke(prompt)
    state["options_json"] = payload.model_dump_json()
    state["messages"].append(AIMessage(content=state["options_json"]))
    return state
```

You can then introduce the agent to reflect and select ([Example 2-4](#)), which prunes the tree. Here, a more powerful LLM evaluates the three proposed options. The model can be instructed to critique each option based on criteria such as clarity and originality, and then select the best one. This step is crucial in ToT, as it ensures the agent does not simply proceed with the first idea but instead critically assesses the alternatives and chooses the most promising path to pursue.

Example 2-4. Pruning Node

```
def reflect_and_select(state: BlogState) -> BlogState:
    """Judge (stronger): critique options and select best one."""
    chooser = judge_llm.with_structured_output(ChoicePayload)
    eval_prompt = (
        "Evaluate the 3 approaches for clarity, originality, developer relevance
        and feasibility under time constraints. Pick ONE by index 0..2 and :
        f"Options JSON:\n{state['options_json']}"
    )
    choice: ChoicePayload = chooser.invoke(eval_prompt)
    opts = json.loads(state["options_json"])[ "options" ]
    idx = max(0, min(choice.choice_index, len(opts)-1))
    choice.choice_index = idx
    state["choice_json"] = choice.model_dump_json()
    state["messages"].append(AIMessage(content=state["choice_json"]))
    return state
```

As before, I omit the rest of the code for the other nodes for brevity, but you can find the complete implementation in the [book's repository](#). What I want to show here is how to tie everything together in LangGraph and which types of LLMs I suggest using. One important paradigm here is the handoff. A *handoff* defines how control shifts between agents: who speaks next, who acts next, and under what conditions. In LangGraph, nodes encode the logic of the agents, and edges represent the handoffs. By wiring multiple nodes together, you are effectively building a MAS within a single graph. The shared state is the memory and context that all nodes operate on. How this wiring looks in practice for ToT is illustrated in [Example 2-5](#).

Example 2-5. Creating the graph for ToT

```
graph = StateGraph(BlogState)
graph.add_node("propose", propose_options) ❶
graph.add_node("reflect", reflect_and_select) ❷
graph.add_node("research", research_with_tools) ❸
graph.add_node("draft", draft_outline_and_intro) ❹

graph.add_edge(START, "propose")
graph.add_edge("propose", "reflect")
graph.add_edge("reflect", "research")
graph.add_edge("research", "draft")
graph.add_edge("draft", END)
```

```
checkpointner = MemorySaver()  
app = graph.compile(checkpointer=checkpointner)
```

- ❶ Small creative generator LLM.
- ❷ Stronger judge/reflector LLM.
- ❸ Tool-using researcher.
- ❹ Small writer model.

When you run the code, you'll see different topics proposed, including reference links. To optimize the results further, you could implement a human-in-the-loop ([“Designing Human-in-the-Loop Workflows”](#)) interaction to review and approve the actions.

It's tempting to think of CoT and ToT as just clever prompting hacks, but you've seen they can be powerful engineering tools. [Figure 2-2](#) contrasts both approaches: on the left a single agent reasoning with CoT, on the right a multi-agent setup where different roles collaborate to explore, reflect, and refine mimicking ToT.

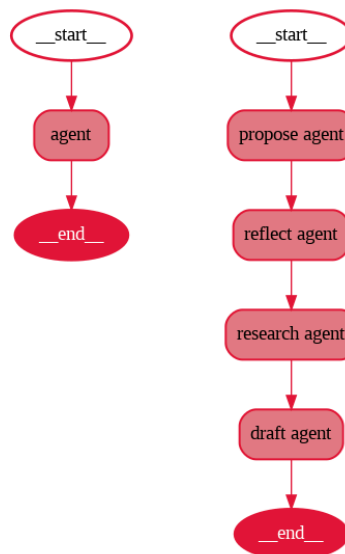


Figure 2-2. Control-flow diagram comparing CoT and ToT.

Together, they show how structured reasoning can range from a linear chain of intermediate steps to a branching process of exploration and selection. Both highlight that when agents are guided to think in stages, whether as a single voice or as a coordinated team, the outcome is more deliberate and reliable.

How to Make Your Agent Take Action and Reflect

You've seen in the previous section that CoT and ToT can be helpful for reasoning, but reasoning alone isn't enough. An agent that only reasons never leaves the page, and an agent that only acts can't learn from its de-

cisions. To combine the two, you need ReAct, which stands for reasoning and action.

Mathematically, you can think of this as mapping a context c_t to an action a_t within an action space $\hat{A} = A \cup L$. Here A is the set of task-specific actions that impact the environment, while L is the set of language-based reasoning traces or thoughts. Each action step therefore produces not just a task action a_t , but also a reasoning trace $\hat{a}_t$.

These traces are folded back into the agent's state so that the context evolves as:

$$c_{t+1} = (c_t, \hat{a}_t)$$

In addition, you follow the policy:

$$\pi(a_t|c_t)$$

Here, $c_t = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t)$. In other words, the agent carries forward not only what it has done, but also why it chose that path. This recursive structure is what enables reflection: the agent's future decisions are informed by both its history of actions and the reasoning it has accumulated. The flow diagram in [Figure 2-3](#) illustrates a minimal agent loop implemented in LangGraph.

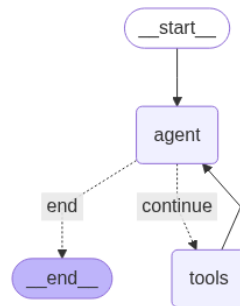


Figure 2-3. Control-flow diagram of a ReAct agent. This loop is the operational form of the policy $\pi(a_t|c_t)$.

The process begins at `__start__`, this hands control to the agent, and then branches depending on the outcome:

- If the agent decides it has reached a conclusion, it produces an end signal and moves to `__end__`.
- If more reasoning or action is required, the agent issues a continue signal, possibly calling tools to gather additional information or perform actions.
- After interacting with tools, control flows back to the agent, allowing it to integrate results and reflect before deciding whether to continue or stop.

This loop is the essence of ReAct: combining reasoning, action, and reflection in an iterative cycle. ReAct is more than “the model thinks and then acts”. The reasoning trace itself becomes part of the evolving state. That

is, the ReAct agent alternates between reasoning traces and tool outputs, and those traces feed back into context.

While this example is in the context of a single agent, the same structure extends naturally into larger MAS. Each agent can maintain its own cycle of reasoning and acting, with reasoning traces shared or exchanged across agents. This makes ReAct not only a tool for individual decision-making, but also a foundation for collaboration and reflection in more complex agentic setups.

Let's make the math and the diagram concrete, and let me show you how this actually maps into code. The following listings turn the ReAct equations into code. Note, as before, I'll just show the important parts here, but the full running implementation will be in the accompanying notebook.

Before we jump into the listings, remember the intent: we carry a growing conversation state c_t while the model alternates between language traces L and task actions A . Each step appends either a thought, an action, or a tool observation back into the state. The code mirrors this loop. Therefore, you first create the state for the graph as in [Example 2-6](#).

Example 2-6. Create ReAct state

```
class AgentState(TypedDict): ❶
    messages: Annotated[Sequence[BaseMessage], add_messages] ❷
```

❶ The state of the agent.

❷ Context c_t is the message stream.

Now you give the model two powers in [Example 2-7](#). It can produce language tokens that act as thoughts L , and it can request structured tool calls A . Binding the tools makes these actions first-class outputs that the runtime can dispatch.

Example 2-7. Init LLM

```
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
model = model.bind_tools(TOOLS) ❶
```

❶ Binding tools to the model augments the action space to $\hat{A} = A \cup L$.

[Example 2-8](#) is the policy step. It reads the entire message history, applies the system rules, and produces exactly one thing: either a thought, or a tool call request, or both. You don't branch here, you simply return what the model decided.

Example 2-8. Call the model

```
def call_model(state: AgentState, config: RunnableConfig) -> Dict[str, Any]:  
    response = model.invoke([SYSTEM_PROMPT] + state["messages"], config) ❶  
    return {"messages": [response]} ❷
```

❶ Policy $\pi(c_t) \rightarrow \hat{a}_t$ is `model.invoke` on the current context.

❷ The reducer appends new messages, implementing $c_{t+1} = (c_t, \hat{a}_t)$.

In [Example 2-9](#) you translate intent into effect. Every requested tool call is executed deterministically, and its result is wrapped as a message. By pushing the observation into the same message stream, you let the next policy step reason over fresh evidence without special cases.

Example 2-9. Execute tool calls

```
def tool_node(state: AgentState) -> Dict[str, Any]:  
  
    last = state["messages"][-1]  
    outputs: list[ToolMessage] = []  
    for tc in getattr(last, "tool_calls", []) or []: ❶  
        name = tc.get("name")  
        args = tc.get("args") or {}  
        tool_fn = TOOLS_BY_NAME.get(name)  
        if not tool_fn:  
            result = json.dumps({"error": f"Unknown tool {name}"})  
        else:  
            result = tool_fn.invoke(args)  
        outputs.append(  
            ToolMessage(  
                content=result, ❷  
                name=name,  
                tool_call_id=tc.get("id"),  
            )  
        )  
    return {"messages": outputs}
```

❶ If the model chose an action $a_t \in \mathcal{A}$, execute the tool.

❷ Tool output is the observation that informs the next step.

The stopping rule in [Example 2-10](#) is simple. If the model asked for a tool, it must complete that effect before asking it again. If it didn't, you'd assume it produced a final answer. This keeps the loop predictable and avoids orphaned actions.

Example 2-10. Halting condition

```
def should_continue(state: AgentState) -> str:
    last = state["messages"][-1]
    if getattr(last, "tool_calls", None): ❶
        return "continue"
    return "end"
```

❶ Halting test reads the last AI message to decide continue or end.

Now you connect the pieces into a small state machine, as shown in [Example 2-11](#). The edges encode the control flow that mirrors the ReAct diagram. The policy node runs first. If it emits an action, we route to the tools node and then back to policy. If it emits only a thought or a final answer, we terminate.

Example 2-11. Build workflow graph

```
workflow = StateGraph(AgentState)
workflow.add_node("agent", call_model)
workflow.add_node("tools", tool_node)
workflow.set_entry_point("agent")

workflow.add_conditional_edges(
    "agent",
    should_continue,
    {"continue": "tools", "end": END},
)

workflow.add_edge("tools", "agent")
graph = workflow.compile()
```

At runtime, the graph advances one edge at a time. The conditional edge decides whether the next hop is `tools` or `END`. The fixed edge from `tools` back to `agent` completes the think-act loop.

To see this in action, you prompt a question, I asked how the weather will be in Zurich. The LLM first decides which tool to call to answer best, then the tool returns evidence, and the model writes a final answer grounded in that evidence. Because every step is recorded in the message stream, the reasoning is retrievable:

```
> Entering new ReAct trace...
Question: What is the weather in Zurich today?

Final Answer: What is the weather in Zurich today?

> Finished trace.

Reasoning Summary: selecting a tool based on the query
```

```

Action:
```

{
 "action": "internet_search",
 "action_input": {
 "query": "Zurich weather today"
 }
}
```

Observation:
```

{
 "title": "Zurich, Zurich, Switzerland Weather Forecast",
 "snippet": "Hourly Weather \u00b7 1 PM 74\u00b0. rain drop 0% \u00b7 2 PM 80\u00b0. rain drop 0% \u00b7 3 PM 82\u00b0. rain drop 0% \u00b7 4 PM 83\u00b0. rain drop 0% \u00b7 5 PM 84\u00b0. rain drop 0% \u00b7 6 PM ...
```

Final Answer: Today in Zurich, the weather is mostly sunny with a high of 82°F (about 28°C) and a low of 62°F (about 17°C). Winds are light from the southwest at 6 mph.

For more details, you can check the full forecast [here](https://www.accuweather.com/en/ch/zurich/316622/weather-forecast/316622)

> Finished trace.

```

Now you understand how powerful ReAct can be. It's not just that the agent alternates between thinking and acting, but that the reasoning itself becomes part of the state. This recursive process enables agents to adapt, refine their strategies, and explain their choices. In practice, it allows even relatively small models to perform complex tasks more reliably, while making their decision-making process transparent.

Together, CoT, ToT, and ReAct illustrate how different reasoning strategies can shape the intelligence of your agents. Each approach has its strengths: CoT offers transparency and reliability, ToT explores multiple alternatives before committing, and ReAct closes the loop by combining reasoning with tool use. [Table 2-3](#) gives you a consolidated overview of these reasoning patterns along with tips for when they are most useful.

Table 2-3. Pattern summary: Structured reasoning methods (CoT, ToT, ReAct)

Method	Use Case	Benefits	Tradeoffs
Chain of Thought (CoT)	Step-by-step reasoning for math, planning, analysis	Improves reliability and transparency; enables smaller models on harder tasks	Slower, verbose outputs; sometimes redundant
Tree of Thought (ToT)	Exploratory tasks with multiple solution paths (creative writing, strategy search)	Explores alternatives; reduces “first-answer bias”	Higher compute cost; requires orchestration (selection and pruning)
ReAct	Tasks needing both reasoning and tool use (research, coding, APIs)	Integrates reasoning + action; creates audit trail; adaptive to feedback	More complex state management; risk of over-calling tools

This highlights how these methods can become powerful building blocks once you integrate them into your overall agentic workflow.

Surely You’re Joking, AI Agent!

Sometimes an agent’s behavior can be entertaining. It might call the wrong tool, loop endlessly, or produce an oddly creative response. In those moments you might laugh and say to yourself addressing the AI agent: “You’re joking, right?” But when the same unpredictability reaches into access control, financial operations, or customer data, it’s no longer a joke. An action taken at the wrong moment or on the wrong system can have serious consequences. This is why *human-in-the-loop* (HITL) exists. HITL allows the agent to pause at critical points, ask for your approval, and continue only once it’s safe to proceed.

HITL is not only about catching errors before they happen. It also provides accountability and transparency. By inserting checkpoints into an agent’s workflow, you make sure irreversible actions are reviewed, decisions are traceable, and sensitive operations stay under human control. Far from slowing the system down, this oversight makes agents more trustworthy and easier to adopt in production settings.

Designing Human-in-the-Loop Workflows

There are a couple of different interruptions that you can perform with a HITL workflow. The most common ones are shown below.

Approve or reject

Pause the graph before a critical step, such as an API call, to review and approve the action. If the action is rejected, you can prevent the graph from executing the step, and potentially take an alternative action. This pattern often involve routing the graph based on the human's input.

Review and edit

Pause the graph to review and edit the graph state. This is useful for correcting mistakes or updating the state with additional information.

Review tool calls

If a tool call could affect external systems, inspect its arguments before execution. For example, you might confirm booking details before a travel reservation is made.

Validate human input

Explicitly request human input at a particular step in the graph. This is useful for collecting additional information or context to inform the agent's decision-making process or for supporting multi-turn conversations.

LangGraph offers an [extensive documentation](#) on how to apply these patterns in your own applications. In addition, the [book's repository](#) includes complete, runnable examples of six different patterns. You can select among them through a `main()` function, as shown in [Example 2-12](#).

Example 2-12. Main function for HITL pattern demos

```
def main():
    menu = """
Pick a demo
1. Human feedback loop for writing
2. Approval gate before API call
3. Review and edit state
4. Parallel interrupts with resume map
5. Tool call review in a tiny ReAct loop
6. Static interrupts for debugging
q. Quit
> """
```

In the following code examples ([Example 2-13](#) - [Example 2-20](#)) I'll explain how to implement the patterns covered above and also cover the main points you'd want to watch out for when you implement them.

Combines control flow (edges) and state updates (nodes). It facilitates multi-actor (or multi-agent) communication. For example, a node can update its state and decide the next node to visit. LangGraph enables this by returning a Command object from node functions.

Use approve or reject when you want to gate a side-effecting step like an HTTP call.

[Example 2-13](#) asks the LLM to produce a request plan and stores it in `proposed_request`.

Example 2-13. Propose request with LLM

```
def b_propose(state: BState) -> BState:
    prompt = "Return only JSON with keys url and params for GET to\n"
            "https://httpbin.org/get using q and limit."
    text = LLM.invoke(prompt).content ❶
    m = re.search(r"\{.*\}", text, re.S)
    data = {"url": "https://httpbin.org/get", "params":
            {"q": "fallback", "limit": 1}} ❷
    if m:
        try:
            data = json.loads(m.group(0)) ❸
        except Exception:
            pass
    return {"proposed_request": data} ❹
```

- ❶ Ask the LLM to generate a structured plan.
- ❷ Fallback default request if parsing fails.
- ❸ Try to parse JSON out of the model's output.
- ❹ Store result in state under `proposed_request`.

[Example 2-14](#) calls `interrupt(...)` to pause and request human approval or a revision.

Example 2-14. Approval gate for request

```
def b_gate(state: BState) -> Command[Literal["b_call", "b_propose"]]:
    v = interrupt({ ❶
        "question": "Approve or revise request",
        "proposed_request": state["proposed_request"], ❷
        "schema": {
            "type": "object",
            "properties": {
                "action": {"enum": ["approve", "revise"]}, ❸
                "update": {"type": "object"}
```



```

        },
        "required": ["action"]
    }
})
action = v.get("action")
if action == "approve":
    return Command(goto="b_call", update={"decision": "approved"}) ❹
upd = v.get("update") or {}
new_req = state["proposed_request"].copy()
if "url" in upd:
    new_req["url"] = upd["url"]
if isinstance(upd.get("params"), dict):
    new_req.setdefault("params", {}).update(upd["params"])
return Command(goto="b_gate", update={"proposed_request": new_req,
"decision": "revised"}) ❺

```

- ❶ Pause execution and wait for human decision.
- ❷ Present the proposed request for inspection.
- ❸ Enforce structured approval/revision schema.
- ❹ On approval, route to the call step.
- ❺ On revision, loop back to the gate with updated request.

On `resume` you pass `Command(resume={"action": "approve"})` to continue, or `{"action": "revise", "update": {...}}` to merge edits and loop back to the gate.

[Example 2-15](#) runs the HTTP call only after approval and stores `api_result`. You can reject to prevent execution, and routing depends on human input.

Example 2-15. Execute approved HTTP call

```

def b_call(state: BState) -> BState:
    import requests
    r = requests.get(state["proposed_request"]["url"],
params=state["proposed_request"]["params"], timeout=10) ❶
    return {"api_result": {"status_code": r.status_code, "url": r.url}} ❷

```

- ❶ Run the external request only after approval.
- ❷ Store result under `api_result` in state.

You use review and edit to correct or improve model output by writing directly to the state. Here, [Example 2-16](#) generates a short summary with the LLM and stores it in `summary`.

Example 2-16. Generate summary text

```
def c_write(state: CState) -> CState:
    text = LLM.invoke("Write 2 sentences about why human
                        in the loop matters for agents").content ❶
    return {"summary": text} ❷
```

❶ Ask the LLM to produce a candidate summary.

❷ Store summary in state for later review.

[Example 2-17](#) pauses with `interrupt(...)` that includes a JSON schema asking for `edited_text`.

Example 2-17. Interrupt for human edit

```
def c_edit(state: CState) -> CState:
    res = interrupt({ ❶
        "task": "Edit the summary text",
        "summary": state["summary"], ❷
        "schema": {
            "type": "object",
            "properties": {"edited_text": {"type": "string"}}, ❸
            "required": ["edited_text"]
        }
    })
    return {"summary": res["edited_text"]} ❹
```

❶ Pause and request edit input from human.

❷ Provide the generated summary for inspection.

❸ Restrict edits to a string field.

❹ Overwrite summary with human revision.

You resume with `Command(resume={"edited_text": "your revision"})`. The node restarts and returns your edit instead of pausing, then the graph updates summary with the human version. Meaning, you literally pause to review then overwrite the graph state with the edited content.

You use review tool calls when a tool may affect external systems and needs inspection of arguments. [Example 2-18](#) wraps any tool with an `interrupt([...])`. The interrupt payload asks for accept, edit, or respond.

Example 2-18. Wrap tool with review gate

```
def add_hitl(tool_obj: BaseTool | Any) -> BaseTool:
    if not isinstance(tool_obj, BaseTool):
        tool_obj = tool(tool_obj)

    @tool(tool_obj.name, description=tool_obj.description,
          args_schema=tool_obj.args_schema)
    def wrapped(**tool_input):
        request = [{ ❶
                    "action_request": {"action": tool_obj.name, "args": tool_input},
                    "config": {"allow_accept": True, "allow_edit":
                               True, "allow_respond": True}, ❸
                    "description": "Review this tool call"
                }]
        response = interrupt(request)[0] ❷
        if response["type"] == "accept":
            return tool_obj.invoke(tool_input) ❹
        if response["type"] == "edit":
            new_args = response["args"]["args"]
            return tool_obj.invoke(new_args) ❺
        if response["type"] == "response":
            return response["args"] ❻
        raise ValueError("Unsupported interrupt response type")
    return wrapped
```

- ❶ Wrap original tool call in a review request.
- ❷ Pass tool name and arguments for inspection.
- ❸ Allow human to accept, edit, or respond.
- ❹ Pause until decision is received.
- ❺ Execute original tool call if accepted.
- ❻ Execute tool with revised arguments.
- ❼ Replace tool output with human response.

You can resume with on of:

```
{ "type": "accept" }
{ "type": "edit", "args": { "args": { "query": "weather in NY" } } }
{ "type": "response", "args": "Skip the tool right now" }
```

The wrapper either executes the original tool call, executes with edited args, or returns a human message instead of calling the tool. With that, you inspect and approve tool calls before they hit the outside world.

Never put side-effecting code (API calls, DB writes) in the same node as `interrupt(...)`. Always pause first, then execute effects only after explicit approval to avoid repeats when resuming.

There might be also the occasion, that you'd like to get parallel inputs with a single resume. In [Example 2-19](#) you see how you can implement this.

Example 2-19. Parallel human interrupts

```
class DState(TypedDict, total=False):
    text_1: str
    text_2: str

def d_h1(state: DState):
    v = interrupt({"text_to_revise": state["text_1"]}) ❶
    return {"text_1": v} ❷

def d_h2(state: DState): ❸
    v = interrupt({"text_to_revise": state["text_2"]})
    return {"text_2": v} ❹

def build_graph_D():
    g = StateGraph(DState)
    g.add_node("d_h1", d_h1) ❺
    g.add_node("d_h2", d_h2)
    g.add_edge(START, "d_h1") ❻
    g.add_edge(START, "d_h2")
    g.add_edge("d_h1", END)
    g.add_edge("d_h2", END)
    return g.compile(checkpointer=CHECKPOINTINTER) ❼
```

- ❶ Interrupt for first text input.
- ❷ Update state with revised `text_1`.
- ❸ Interrupt for second text input.
- ❹ Update state with revised `text_2`.
- ❺ Add both nodes to the graph.
- ❻ Both nodes branch directly from START.
- ❼ Compile with checkpointer so both interrupts can resume.

Here, both nodes start from `START` and each calls `interrupt(...)`. You use [Example 2-20](#) to build a dictionary that maps `interrupt_id` to its value, and then resume with `Command(resume=that_map)`.

Example 2-20. Resume multiple interrupts

```
def wait_for_interrupt_and_prompt(app, cfg):
    state = app.get_state(cfg)
    ints = getattr(state, "interrupts", []) or [] ❶
    if not ints:
        print("No interrupts pending")
        return None

    if len(ints) > 1: ❷
        print("\nMultiple interrupts pending:")
        for i, it in enumerate(ints, 1):
            print(f"[{i}] id={it.interrupt_id} value={jdump(it.value)}" ❸)
        print("Enter values per interrupt. Leave blank to echo original.")
        resume_map = {}
        for it in ints:
            val = input(f"Value for {it.interrupt_id}: ").strip() ❹
            if val:
                try:
                    resume_map[it.interrupt_id] = json.loads(val) ❺
                except Exception:
                    resume_map[it.interrupt_id] = val
            else:
                resume_map[it.interrupt_id] = it.value ❻
        return Command(resume=resume_map) ❼
```

- ❶ Collect any pending interrupts from state.
- ❷ Handle multiple interruptions at once.
- ❸ Print each interrupt ID and its current value.
- ❹ Ask the human for a new value per interrupt.
- ❺ Try to parse as JSON for structured edits.
- ❻ Keep the original value if no input is given.
- ❼ Resume execution with a full map of updated values.

As you’ve seen, there isn’t a single “right” way to add human oversight. You can choose from different patterns depending on the sensitivity of the task and the amount of control you want to keep. I encourage you to experiment with these patterns in the provided notebook, as this makes it easier to see how human oversight can evolve from simple approval gates into richer collaborations between human and agents.

Advanced Agent Paradigms:

Supervisor and Hierarchical Agents

If your tasks start getting more complex than the ones you’ve seen so far, or if you notice your agent struggling, it can help to break the task into smaller subtasks. Think of it like building a team: each person brings a different skill to the table, and together they solve the larger challenge.

In the same way, you can introduce smaller, independent agents that form a MAS. One common setup is the *supervisor architecture*, where every agent reports to a single supervisor that decides who should act next. A more advanced version is the *hierarchical architecture*, where you don’t just have one supervisor, but supervisors of supervisors. This generalization allows for more flexible control flows.

The diagram in [Figure 2-4](#) shows how a single-agent setup compares to these two multi-agent patterns.

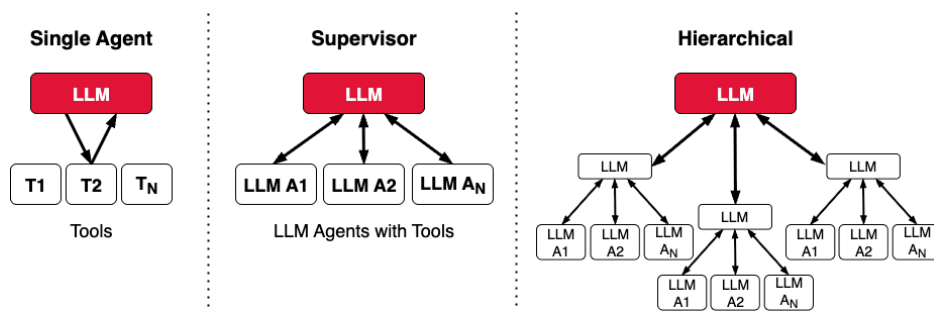


Figure 2-4. Comparison of single, supervisor, and hierarchical agent architectures.

The primary benefits of using a MAS are its modularity, specialization, and control. By separating functionality across agents, you make the overall system easier to develop, test, and maintain. Each agent can be designed as a domain expert, which boosts the system’s performance as a whole. And unlike simple function calling, a multi-agent setup also gives you explicit control over how agents communicate with one another.

Building a Hierarchical Research Team

To make the idea of supervisors and sub-agents concrete, let’s walk through an example where you build a hierarchical research team. I’ll use LangGraph’s prebuilt, reusable `create_react_agent` component to fast-track the implementation of a ReAct agent. Each worker agent then gets a clear role: some focus on web search, others scrape pages, query [Exa](#) for more structured search results, or even look up patents on Google. A supervisor agent sits on top, coordinating the specialists and deciding who should act next. The end goal is to assemble a market research team. [Figure 2-5](#) shows the final application you’re building.

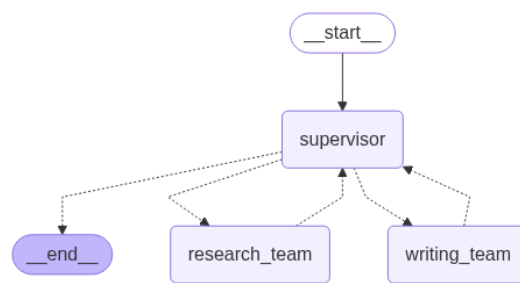


Figure 2-5. Final application with a hierarchical architecture.

But before you get there, you'll start smaller by building your first supervisor setup: the research team. You see the overview of the team you create in [Figure 2-6](#). Once that's in place, the second step is to add a writing team. This group will take the output from the research team and save it to your file system as a simple text file.

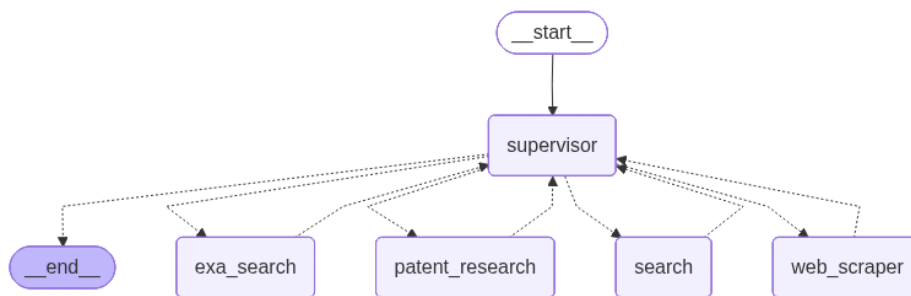


Figure 2-6. Flow diagram for the research team.

I'll leave out the details of how to create tools, since you already learned this in chapter one ([Example 1-2](#)).

The first thing you need is a helper ([Example 2-21](#)) to create your supervisor. This function acts as a router (see [“Mapping FSM/HSM to Agent Frameworks”](#)) that decides which worker should take the next step, or whether the process should finish.

Example 2-21. Helper function for supervisor

```

class State(MessagesState): ❶
    next: str

def make_supervisor_node(llm: BaseChatModel, members: list[str]) -> str: ❷
    options = ["FINISH"] + members
    system_prompt = ( ❸
        "You are a supervisor tasked with managing a conversation between the"
        f" following workers: {members}. Given the following user request,"
        " respond with the worker to act next. Each worker will perform a"
        " task and respond with their results and status. When finished,"
        " respond with FINISH."
    )

    class Router(TypedDict):
        """Worker to route to next. If no workers needed, route to FINISH."""

        next: Literal[*options]

```

```

def supervisor_node(state: State) -> Command[Literal[*members, "__end__"]]
    """An LLM-based router."""
    messages = [
        {"role": "system", "content": system_prompt},
    ] + state["messages"]
    response = llm.with_structured_output(Router).invoke(messages) ❷
    goto = response["next"]
    if goto == "FINISH": ❸
        goto = END

    return Command(goto=goto, update={"next": goto})

return supervisor_node

```

- ❶ Create the state to keep track of the next worker.
- ❷ Define a helper that builds a supervisor node for any set of agents.
- ❸ Provide a system prompt that tells the LLM how to act as the supervisor.
- ❹ Use structured output to reliably decide which worker to call next.
- ❺ Allow the workflow to terminate cleanly when the supervisor says `FINISH`.

Next, you need a helper to create your worker nodes, how to implement this is shown in [Example 2-22](#). This makes the setup concise and reusable, so you don't have to repeat the same boilerplate each time you add a new agent.

Example 2-22. Helper function for node creation

```

def make_react_worker_node(
    *,
    llm: ChatOpenAI,
    name: str,
    tools: list,
    prompt: str | None = None,
    goto: str = "supervisor",
):
    agent = create_react_agent(llm, tools=tools, prompt=prompt) ❶

    def node(state: State) -> Command[Literal["supervisor"]]:
        result: Dict[str, Any] = agent.invoke(state) ❷
        msgs: Sequence[BaseMessage] = result.get("messages", [])
        content = getattr(msgs[-1], "content", "") if msgs else ""
        return Command( ❸
            update={"messages": [HumanMessage(content=content, name=name)]},
            goto=goto,
        )

```



```
return node
```

- ❶ Create a specialized ReAct agent with its tools and role prompt.
- ❷ Invoke the agent with the current state.
- ❸ Pass the agent's latest output back into the state and return to the supervisor.

Once you have these helpers, you can move on to defining your agents. It's helpful to give each agent a specific role. Since the goal here is to build a market research team, the roles should be very targeted. My role prompts are shown in [Example 2-23](#).

Example 2-23. Assign roles to research agents

```
SEARCH_PROMPT = """Role: Web researcher. Use the search tool and return a
concise research note with sources. No follow-up questions."""

SCRAPER_PROMPT = """Role: Web scraper. Use the scraping tool to fetch details
from given URLs and summarize key findings. No follow-up questions."""

EXA_PROMPT = """Role: Research assistant. You can search for all recent info
on Exa Search. Your response should clearly articulate the key points you fo

PATENT_PROMPT = """Role: Market researcher with 20 years of experience.
You are very knowledgeable in patent research and in finding up-to-date info
about patents using the Google Patents API."""
```

Now, let's create the agents. Using the helper from [Example 2-22](#), you can instantiate each worker and assign it the appropriate role and tool. The implementation is shown in [Example 2-24](#).

Example 2-24. Create research agents

```
specs = [
    dict(
        name="search",
        tools=[tavily_tool],
        prompt=SEARCH_PROMPT,
    ), ❶
    dict(
        name="web_scraper",
        tools=[scrape_webpages],
        prompt=SCRAPER_PROMPT,
    ),
    dict(
        name="exa_search",
        tools=[exa_search_tool],
        prompt=EXA_PROMPT,
    ),
]
```

```

dict(
    name="patent_research",
    tools=[patent_search],
    prompt=PATENT_PROMPT,
),
]

nodes = {s["name"]: make_react_worker_node(llm=llm, **s) for s in specs} ❷

search_node = nodes["search"]
web_scraper_node = nodes["web_scraper"]
exa_search_node = nodes["exa_search"]
patent_research_node = nodes["patent_research"]

```

- ❶ Define each worker with its tools and role.
- ❷ Use the helper to instantiate all workers without repeating boilerplate.

To complete the supervisor architecture, we need to set up a supervisor that can coordinate across all four research agents. This ensures the workflow doesn't stall and that each agent contributes at the right time. See [Example 2-25](#) for the implementation.

Example 2-25. Create research supervisor node

```

research_supervisor_node = make_supervisor_node(
    llm, ["search", "web_scraper", "exa_search", "patent_research"]
) ❶

```

- ❶ Create a supervisor that coordinates across all four research agents.

Finally, we need to build the actual graph by registering nodes and connecting the edges that define how control flows between them. This results in a fully functioning research team graph, as shown in [Example 2-26](#).

Example 2-26. Build research team graph

```

research_builder = StateGraph(State)

research_builder.add_node("supervisor", research_supervisor_node) ❶
research_builder.add_node("search", search_node)
research_builder.add_node("web_scraper", web_scraper_node)
research_builder.add_node("exa_search", exa_search_node)
research_builder.add_node("patent_research", patent_research_node)

research_builder.add_edge(START, "supervisor") ❷
research_builder.add_edge("search", "supervisor")
research_builder.add_edge("web_scraper", "supervisor")

```

```
research_builder.add_edge("exa_search", "supervisor")
research_builder.add_edge("patent_research", "supervisor")

research_graph = research_builder.compile() ❸
```

- ❶ Register supervisor and worker nodes.
- ❷ Connect edges so all workers route back to the supervisor.
- ❸ Compile the graph into an executable workflow.

To test this setup, you can run the code from [Example 2-27](#).

Example 2-27. Run research team with prompt

```
for s in research_graph.stream(
    {"messages": [ ("user", ""What are AI agents? Are there any patents out t
                        about LLM agents?"" ) ]}, ❶
    {"recursion_limit": 100},
):
    print(s)
    print("---")
```

- ❶ Run the research graph on a query; the supervisor orchestrates which agents act.

This returns a short description AI agents and then uses the patent tool to check for recent patents. The steps look like this (I’ve left out the actual messages between agents for brevity):

```
{'supervisor': {'next': 'search'}}
{'supervisor': {'next': 'patent_research'}}
{'supervisor': {'next': '__end__'}}
```

You can set up your writing team in the same way. The notebook for this section includes the full working code, but [Figure 2-7](#) gives you a quick overview of what the writing team looks like.

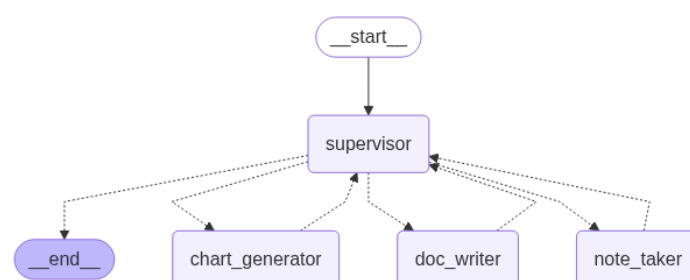


Figure 2-7. Flow-chart for writing team.

This modular approach makes it easy to compose increasingly capable MAS step by step: first by wiring up specialized workers, then by combin-

ing entire teams into larger graphs. In practice, this means you can start small—say with search, scraping, and patent lookup in a research team, and then plug that into a writing team to form a full end-to-end pipeline, or extend it even further with internal data and chart writing and plotting agents.

To move from individual teams to a proper hierarchy, you'll need to build another graph that ties everything together. Think of this as your “super graph,” shown in [Example 2-28](#).

Example 2-28. Build hierarchical architecture

```
super_builder = StateGraph(State)
super_builder.add_node("supervisor", teams_supervisor_node)
super_builder.add_node("research_team", call_research_team)
super_builder.add_node("writing_team", call_paper_writing_team)

super_builder.add_edge(START, "supervisor")
super_graph = super_builder.compile()
```

Once that's in place, you can prompt your hierarchical MAS as shown in [Example 2-29](#).

Example 2-29. Run hierarchical MAS

```
TARGET_FILE = "semiconductor_whitepaper.txt"

TASK_MSG = f"""
Write an 800-word research report white paper on semiconductor development.
Start with an executive summary of your findings.
Search for relevant recent patents and include links to them.
IMPORTANT: Provide links to all your sources.
Finally, save the full report to disk as a .txt file using the write_document
Use file_name="{TARGET_FILE}".
"""

for step in super_graph.stream(
    {
        "messages": [
            ("user", TASK_MSG.strip())
        ],
    },
    {"recursion_limit": 150},
):
    print(step)
    print("---")
```

Running this setup will result in the following sequence of supervisor decisions (messages again omitted):

```
{ 'supervisor': { 'next': 'research_team' } }
{ 'supervisor': { 'next': 'writing_team' } }
{ 'supervisor': { 'next': 'writing_team' } }
{ 'supervisor': { 'next': '__end__' } }
```

At the end of the run, the report is successfully saved to `semiconductor_whitepaper.txt`. In practice, this workflow takes only about two minutes to gather the research, generate the text, and write everything to file.

Developing a Swarm of Agents

Another important paradigm is the swarm architecture. Here, agents handoff control to one another dynamically based on their specializations. Unlike a strict hierarchy, swarms emphasize peer-to-peer collaboration. The system keeps track of which agent was last active so that subsequent interactions continue seamlessly with the right one. This back-and-forth exchange allows work to bounce naturally between agents, such as when a researcher gathers sources and then hands them to a writer, who may in turn request more data before finishing the synthesis. [Figure 2-8](#) illustrates this continuous flow.

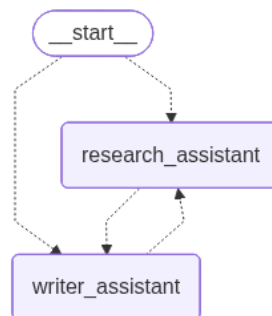


Figure 2-8. Handoff flow between a swarm of agents.

LangGraph provides an [open-source library](#) to fast track the development of swarms of agents. By default, the agents in the swarm use handoff tools created with the prebuilt `create_handoff_tool`. You can also create your own custom handoff tools to better fit your workflow. For example, you might:

- Change the tool name or description to make the handoff purpose clearer.
- Add tool call arguments for the LLM to populate, such as a specific task description for the next agent.
- Control what data is passed to the next agent. By default, `create_handoff_tool` passes the full message history (all prior agent messages) and a tool message confirming the handoff, but you may choose to send only a filtered subset of context.

Before wiring up a full swarm, you first need to define the handoff helpers as shown in [Example 2-30](#). These tools act as signals that let one agent pass control to another. In the example below, the research assistant can handoff to the writer once enough sources are collected, and the writer can hand control back if more evidence is needed.

Example 2-30. Create handoff tools

```
to_writer = create_handoff_tool( ❶
    agent_name="writer_assistant",
    description="""Transfer to the writing assistant to synthesize sources in
        an answer."""
)
to_research = create_handoff_tool( ❷
    agent_name="research_assistant",
    description="""Return to the research assistant to fetch or scrape more
        sources."""
)
```

- ❶ Defines a tool that lets the research assistant handoff to the writing assistant.
- ❷ Defines the reverse tool so the writer can send control back to the research assistant.

Once you have the handoff helpers in place, you can move on to defining your agents. It's helpful to keep the roles very distinct. In this setup you want one agent focused purely on research, and the other focused on writing. The handoff tools ensure the workflow can bounce back and forth when more sources are needed or when it's time to synthesize. The roles are shown in [Example 2-31](#).

Example 2-31. Assign roles to research and writing assistants

```
research_assistant = create_react_agent( ❶
    model=llm,
    tools=[tavily_tool, scrape_webpages, to_writer], ❷
    prompt=(
        """You are a research assistant. Search the web with Tavily.
        When you have 3 to 5 solid sources, scrape key pages for details,
        then hand off to the writer assistant."""
    ),
    name="research_assistant",
)

writer_assistant = create_react_agent( ❸
    model=llm,
    tools=[to_research], ❹
    prompt=(
        """You are a writing assistant. Read the provided Documents and messsage.
        Synthesize a concise answer with citations by site name in brackets.
        """
    )
)
```

```

        sources are thin or unclear, hand back to research with a short request.
    ),
    name="writer_assistant",
)

```

- ❶ Defines the research assistant, responsible for web search and scraping.
- ❷ Grants the research assistant access to its tools, including the hand-off to the writer.
- ❸ Defines the writing assistant, responsible for synthesis and citation.
- ❹ Allows the writer to hand control back to the research assistant if more evidence is needed.

Now that both roles are defined, we can wire them into a swarm. The swarm provides the coordination mechanism: it starts with the research assistant by default, but allows handoff to the writer and back again until the task is complete. The implementation is shown in [Example 2-32](#).

Example 2-32. Build research–writer swarm

```

swarm = create_swarm(
    agents=[research_assistant, writer_assistant],
    default_active_agent="research_assistant",
).compile()

```

To test the system, you can run the following example prompt. Here, I requested a summary of fintech licensing in Switzerland. The swarm first searches for sources, then hands off to the writing assistant for synthesis, and if needed returns to research for more data. See [Example 2-33](#) for the prompt.

Example 2-33. Run research–writer swarm

```

user_request = { ❶
    "messages": [
        {
            "role": "user",
            "content": (
                ""Find the current Swiss fintech licensing options for small
                startups. Gather authoritative sources and produce a short
                summary with three bullet points and references.""
            ),
        },
    ],
}

for chunk in swarm.stream(user_request): ❷

```

```
print(chunk)
print()
```

- ❶ Defines a user prompt asking for fintech licensing details in Switzerland.
- ❷ Streams the results from the swarm, showing the handoff-driven workflow in action.

The run produces three clean bullet points with references, citing official Swiss regulator pages and trusted financial sources. However, if the writer finds the evidence too thin, it can trigger a return handoff to re-search. The full cycle takes just about 30 seconds to gather the information and generate the final structured output.

The supervisor, hierarchical, and swarm architectures from this section show how different coordination strategies affect MAS. Supervisors provide centralized control, hierarchies allow scalable organization into teams of teams, and swarms emphasize flexible peer-to-peer collaboration. [Table 2-4](#) compares these patterns and highlights when to use each one.

Table 2-4. Pattern summary: Multi-agent coordination strategies

Architecture	When to Use	Benefits	Tradeoffs
Supervisor	Small teams of agents with clear task boundaries	Simple setup; strong control; avoids deadlocks	Central bottleneck; single point of failure
Hierarchical	Larger, multi-team workflows requiring scalability	Modular; scalable; mirrors organizational structures	Harder to debug; requires careful design
Swarm	Dynamic collaboration between peers (e.g., research ↔ writer)	Flexible; natural back-and-forth flow	Less predictable; requires robust handoff design

This shows how each coordination pattern supports different scales and styles of agent collaboration. By mixing and matching them, you can shape agent workflows that balance control, scalability, and adaptability within your overall MAS design.

Conclusion

The big takeaway from this chapter is that an AI agent’s intelligence comes also from its architecture, not just from the LLM it uses. In light of this, you explored how architectural patterns shape the intelligence and

reliability of AI agents. By moving beyond single-step execution, you saw how reasoning loops like CoT, ToT, and ReAct embed structure into the agent's thought process, making outputs more deliberate and transparent.

Equally important, you saw how human-in-the-loop can make your agents more secure. Interrupts, approval gates, and review steps demonstrated how guardrails make agents trustworthy in real-world settings where errors carry real consequences. The balance between autonomy and oversight is a defining theme of modern agent design.

You also learned how these reasoning methods extend into MAS. Supervisor and hierarchical architectures distribute tasks across specialized workers, while swarm setups enable flexible handoffs between peers. Together, these paradigms show that agent intelligence is not just about stronger models, but about how those models are orchestrated into systems that can plan, act, and reflect.

With these foundations in place, you are ready to move to the next level. In the following chapter, you will explore advanced paradigms that extend these architectures toward scalable execution, deeper planning strategies, and more sophisticated reasoning methods. This will show you how to design agents that can not only react and reflect, but also plan at scale and coordinate complex workflows across larger systems.

- ¹ Jason Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", <https://arxiv.org/abs/2201.11903> (2023).
- ² Shunyu Yao et al. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models.", <https://arxiv.org/abs/2305.10601> (2023).